List of Tables

List of Figures

Table of Contents

1. Abstract

Rapid global urbanization and the growth of modern megacities present unprecedented challenges in municipal administration, transportation management, emergency service response, public utility distribution, and civic resource allocation. Conventional municipal administration frameworks rely on fragmented, legacy systems that struggle to manage high-volume, real-time data feeds or compute optimal resource distributions. Addressing these urban bottlenecks requires integrating core Data Structures and Algorithms (DSA) with modern, multi-tier cloud web technologies to create automated, data-driven, and resilient smart city infrastructure.

The Smart City Management System (SCMS) is an enterprise-grade, full-stack urban orchestration platform designed to model, simulate, optimize, and manage municipal operations in real time. The platform unifies eight dedicated core modules into a single control terminal: a Citizen Directory and Identity Registry; a Municipal Department Operations and Expenditure Analytics module; a Real-Time Traffic Simulator and Congestion Control engine; a Route Pathfinder and Navigation Engine; an Emergency Incident Dispatch Control Room; a Utility Grid Layout Optimizer; a Smart Parking and Public Venue Booking Queue Manager; and an Executive Real-Time Monitoring Dashboard.

To achieve high computational performance and mathematical correctness across these modules, SCMS incorporates multiple fundamental data structures and graph algorithms, including Dijkstra's Shortest Path Algorithm, the A* (A-Star) heuristic search algorithm, Kruskal's Minimum Spanning Tree (MST) algorithm with Disjoint Set Union-Find, and an array-backed First-In-First-Out (FIFO) queue for fair civic resource allocation.

SCMS is built using a decoupled micro-architecture: a React 18 + Vite client frontend utilising Leaflet.js interactive maps and Socket.io WebSocket listeners; a Node.js + Express API middleware server managing JWT security, MongoDB persistence, and traffic simulation ticks; and a high-performance Java Spring Boot 3 algorithmic engine. To ensure uninterrupted operation, the backend features an automatic dual-engine fallback — if the Java Spring Boot service is offline or sleeping, an internal JavaScript algorithm engine executes seamlessly, guaranteeing continuous service availability.

Empirical evaluation confirms that SCMS completes path calculations in sub-millisecond execution times (0.036 ms to 0.049 ms), reduces explored search nodes by up to 25% using A* heuristics, computes minimum-cost spanning trees deterministically with a 71.4% reduction in installation cost compared to a fully connected topology, and handles real-time traffic ticks smoothly at a two-second refresh interval. SCMS therefore provides a practical, scalable, and academically rigorous model for modern intelligent transportation and smart city infrastructure, demonstrating how classical computer science theory can be operationalised inside a production-grade web platform.

Keywords:

Smart City, Data Structures and Algorithms, Dijkstra's Algorithm, A Search, Kruskal's Minimum Spanning Tree, Graph Theory, Real-Time Systems, MERN Stack, Spring Boot, WebSockets, Urban Informatics.*

2. Introduction

2.1 Background and Urbanization Problem Statement

Modern cities represent complex, interconnected networks of human population, physical infrastructure, public utilities, and transportation corridors. As urban populations continue to expand rapidly, municipal authorities face significant operational challenges. Unplanned urban growth leads to severe traffic congestion, delayed emergency vehicle response, unbalanced utility loading, inefficient waste collection, and backlogged municipal service requests.

Traditional municipal management relies on siloed software applications or manual record-keeping, creating significant communication lag between municipal departments. For example, emergency response units often navigate through heavy traffic without real-time congestion awareness, while utility expansion planning frequently results in redundant piping or wiring paths due to a lack of network optimisation tools.

Solving these multi-faceted urban problems requires a unified computational approach. By applying Data Structures and Algorithms — such as weighted graphs, shortest-path search algorithms, minimum spanning trees, priority queues, and linear waitlists — city operations can be automated, optimised, and visualised within a single real-time platform.

2.2 Role of Data Structures and Algorithms in Modern Smart Cities

Computer science principles form the backbone of modern intelligent transportation systems (ITS) and smart city architectures:

- **Graph Theory:** Road intersections and utility substations are modelled as vertices (nodes), while connecting roads, power lines, or water mains are modelled as weighted edges. This formulation allows complex physical infrastructure to be processed mathematically.
- **Shortest Path Algorithms:** Algorithms such as Dijkstra's and A* enable real-time route optimisation for citizens, logistics providers, and emergency responders, reducing travel time and fuel consumption.
- **Spanning Tree Algorithms:** Kruskal's Algorithm solves structural optimisation problems, enabling municipal engineers to connect distributed utility networks with the minimum possible cable or pipe length.
- **Queueing Theory and Linear Data Structures:** First-In, First-Out (FIFO) queues manage finite urban resources — such as limited parking slots or public facility bookings — ensuring transparent, first-come, first-served access.

2.3 Smart City Management System (SCMS) Overview

The Smart City Management System (SCMS) is an enterprise-grade urban orchestration platform created to model, optimise, and manage smart city operations. Developed as a decoupled multi-tier web application, SCMS provides an interactive interface for both citizens and city administrators. SCMS unifies municipal functions into eight core modules, described in detail in Section 2.4, that together cover identity management, financial oversight, transportation, emergency response, infrastructure planning, and public resource allocation.

2.4 Modular Capabilities Breakdown

2.4.1 Citizen Directory and Identity Registry

Maintains comprehensive, indexed records for city residents across municipal wards (Ward A through Ward H). Supports fast substring searching, multi-criteria filtering (Category: General, OBC, SC, ST, EWS; Status: Active, Inactive, Deceased, Migrated), Aadhaar verification, and historical audit tracking to record every creation, modification, or status change made to a citizen's record.

2.4.2 Municipal Department Operations and Expenditure Analytics

Manages municipal department profiles (Water Supply and Sanitation, Traffic and Public Transport, Power and Energy Grid, Emergency and Disaster Response, Waste Management and Sanitation). Tracks budget allocations, actual expenditures, financial utilisation ratios, and deployed staff counts so administrators can identify departments that are over- or under-utilising allotted funds.

2.4.3 Real-Time Traffic Simulator and Congestion Control

A continuous simulation engine built on Socket.io WebSockets. Every two seconds, the simulator updates vehicle density and average speed across twelve city intersections and seventeen interconnecting road segments, emitting real-time updates to connected clients and rendering interactive congestion heatmaps on a Leaflet.js map.

2.4.4 Route Pathfinder and Navigation Engine

Allows citizens and control-room operators to compute optimal travel paths between city locations (for example, Connaught Place to Chandni Chowk). Supports both Shortest Path (minimum geographic distance) and Fastest Path (traffic-adjusted duration) modes. Displays side-by-side performance benchmarks comparing Dijkstra's Algorithm against A* Search, and generates K-shortest alternative routes with edge penalties for civilian traffic diversion.

2.4.5 Emergency Incident Dispatch Control Room

A centralised emergency logging and dispatch centre. Manages active incidents (fires, vehicle collisions, water-main bursts, power outages) and dispatches emergency units such as FIRE-12 and AMBULANCE-04. Computes priority green-wave routes for emergency vehicles while recalculating alternative bypass routes for civilian traffic in real time.

2.4.6 Utility Grid Layout Optimizer

An automated network-planning tool for municipal electricity substations, water reservoirs, and gas distributaries. Computes the Minimum Spanning Tree (MST) across utility nodes using Kruskal's algorithm, identifying the minimum total cable or pipe length required to interconnect all facilities without redundant loops.

2.4.7 Smart Parking and Public Venue Booking Queue Manager

Manages public facility venue reservations (Town Hall Auditorium, Central Park Sports Complex) and smart parking slot allocations (Zones A through D). Implements an array-backed FIFO queue data structure with O(1) time complexity to guarantee transparent, first-come, first-served queue processing.

2.4.8 Executive Audit and Real-Time Control Dashboard

The central administrative control terminal, displaying aggregate city health metrics, total active emergency incidents, utility grid overload alerts, traffic congestion summaries, and an audit trail log documenting every system decision for accountability and post-incident review.

2.5 Multi-Tier Microservice Architecture

SCMS follows a decoupled, four-tier microservice architecture that separates presentation, orchestration, computation, and persistence concerns. This separation allows each tier to be independently developed, deployed, scaled, and replaced without disrupting the others.

Tier	Technology	Responsibility
Presentation	React 18 + Vite (Vercel)	Interactive UI, Leaflet maps, live charts, WebSocket listeners
API Gateway	Node.js + Express (Render)	REST endpoints, JWT auth, Socket.io broadcast, MongoDB access
Algorithm Engine	Java Spring Boot 3 (Railway)	Dijkstra, A*, Kruskal MST, custom data structures
Persistence	MongoDB Atlas	Citizens, departments, incidents, bookings, sensor logs

Table 2.1 — Four-tier microservice architecture of SCMS

2.6 Educational and Software Engineering Objectives

1. **Algorithm Implementation from Scratch:** Shortest-path and spanning-tree algorithms are implemented directly in Java Spring Boot and JavaScript without reliance on third-party routing black boxes.
2. **Dual-Engine Fault Tolerance:** An automatic backend fallback mechanism ensures that if the Java Spring Boot service is hibernating or unreachable, an in-memory Node.js Dijkstra engine takes over seamlessly, ensuring continuous service uptime.
3. **Interactive Visual Verification:** Integrates Leaflet.js and OpenStreetMap tiles, allowing users and evaluators to observe node traversal, edge relaxation, and path rendering graphically.
4. **Real-World Software Engineering Practice:** The project applies clean architecture, RESTful API design, role-based access control, and CI/CD-ready deployment configuration, mirroring practices used in industrial software teams.

2.7 Problem Statement, Objectives, and Scope

2.7.1 Problem Statement

Municipal authorities currently lack a single, computationally rigorous platform that unifies citizen records, department finances, live traffic conditions, emergency dispatch, utility infrastructure planning, and public resource booking. Existing municipal software is typically departmental, siloed, and devoid of algorithmic optimisation, resulting in slower emergency response, redundant utility expenditure, and unfair allocation of scarce public resources such as parking and venue slots.

2.7.2 Objectives

- Design and implement a unified, multi-tier web platform that models core municipal operations as computable graph and queueing problems.
- Implement Dijkstra's Algorithm and A* Search from first principles to provide verifiably optimal, low-latency route computation for citizens and emergency responders.
- Implement Kruskal's Minimum Spanning Tree algorithm to minimise utility infrastructure installation cost.
- Implement a fair, O(1) FIFO queue for public parking and venue booking allocation.
- Provide a real-time, WebSocket-driven traffic simulation and monitoring layer with sub-second update latency.
- Empirically benchmark all algorithms for correctness, execution time, and memory footprint, and document the results rigorously.

2.7.3 Scope of the Project

The current scope of SCMS covers a representative simulated city (12 road-network intersections, 17 road segments, and a 5-node utility grid) sufficient to validate algorithmic correctness and system integration end-to-end. The scope explicitly excludes integration with live government databases, physical IoT sensor hardware, and production-grade load balancing, all of which are identified as future work in Section 5.3. The project targets an academic and portfolio-demonstration context rather than an operational government deployment.

2.8 Related Work and Comparative Positioning

Existing intelligent transportation and smart-city research prototypes typically address a single vertical concern in isolation — for example, dedicated route-optimisation engines, standalone traffic-simulation dashboards, or isolated utility-network planning tools. SCMS differs by combining several of these verticals inside one authenticated, role-aware platform with a shared real-time data layer, which allows modules to interact (for example, an emergency dispatch route automatically penalising and rerouting civilian traffic).

Capability	Typical Standalone GPS/Routing App	Typical Municipal ERP	SCMS (This Project)
Shortest-path routing	Yes (proprietary, black-box)	No	Yes — Dijkstra & A*, from scratch
Utility infrastructure optimisation	No	Partial (manual planning)	Yes — Kruskal MST
Real-time traffic visualisation	Partial (crowd-sourced)	No	Yes — 2-second WebSocket ticks
Emergency dispatch integration	No	Partial (manual logging)	Yes — automated green-wave routing
Fair public-resource queueing	No	Partial (manual booking)	Yes — O(1) FIFO queue
Open, from-scratch algorithms	No (closed-source)	No	Yes — fully documented & benchmarked

Table 2.2 — Comparative positioning of SCMS against typical standalone routing applications and municipal ERP systems

2.9 System Requirements

2.9.1 Functional Requirements

- The system shall authenticate users and enforce role-based access (Citizen, Operator, Admin) on every protected route.
- The system shall allow searching and filtering of citizen records by name, ward, category, and status.
- The system shall compute the shortest and fastest path between any two road-network nodes using Dijkstra's Algorithm and A* Search, and display both for comparison.
- The system shall compute a Minimum Spanning Tree over a selected set of utility nodes using Kruskal's Algorithm.
- The system shall allow operators to log emergency incidents and automatically compute a priority dispatch route.
- The system shall manage parking and venue booking requests using a strict FIFO queue.
- The system shall broadcast live traffic density and speed updates to all connected clients every two seconds.

2.9.2 Non-Functional Requirements

- **Performance:** Route computation and MST calculation shall complete within single-digit milliseconds for the reference graph sizes used in this project.
- **Availability:** The system shall remain operational for route computation even if the Java algorithm engine becomes unreachable, via automatic fallback.
- **Security:** Passwords shall be hashed at rest, and all production traffic shall be served over HTTPS/WSS.
- **Usability:** The frontend shall present computed routes, MSTs, and traffic conditions visually on an interactive map rather than as raw coordinate data.
- **Maintainability:** Each tier (frontend, API gateway, algorithm engine, database) shall be independently deployable and independently testable.

3. Algorithm and System Design

3.1 Mathematical Graph Formulation and Data Structure Architecture

The city road network and public utility distribution grids are modelled as non-negatively weighted graphs G = (V, E), where V = {v1, v2, ..., vn} is the set of n vertices representing city intersections or utility nodes, E = {e1, e2, ..., em} is the set of m edges representing connecting roads or utility lines, and w: E → R≥0 is the weight function assigning a non-negative distance, travel time, or installation cost to each edge e = (u, v).

Adjacency List Representation

To achieve optimal space efficiency on sparse road graphs where |E| is much smaller than $|V|^2$, SCMS stores the graph structure using an Adjacency List. Each vertex u ∈ V maps to a linked array of adjacent edges:

```
ConnaughtPlace -> [ { target: KarolBagh, weight: 2.5 }, { target: ChandniChowk, weight: 3.8 } ]
KarolBagh      -> [ { target: ConnaughtPlace, weight: 2.5 }, { target: ChandniChowk, weight: 1.9
} ]
ChandniChowk   -> [ { target: ConnaughtPlace, weight: 3.8 }, { target: KarolBagh, weight: 1.9 } ]
```

- **Space complexity:** O(V + E), compared with $O(V^2)$ for an adjacency matrix.
- **Neighbour retrieval time:** O(deg(u)), supporting fast graph traversal for real-time queries.

3.2 Dijkstra's Shortest Path Algorithm

Theoretical Foundation and Greedy Strategy

Dijkstra's Algorithm is a greedy pathfinding algorithm that calculates the shortest path from a starting vertex s ∈ V to all other vertices in a weighted graph with non-negative edge weights (w(e) ≥ 0). It maintains a set of settled nodes whose minimum distance from s is finalised, repeatedly selecting the unsettled vertex with the minimum tentative distance.

Edge Relaxation Principle

For an edge e = (u, v) with weight w(u, v), if the tentative distance to v can be shortened by travelling through u, the distance is updated: if d[u] + w(u, v) < d[v], then d[v] = d[u] + w(u, v) and parent[v] = u.

Step-by-Step Working Mechanism

5. Initialize d[s] = 0 and d[v] = infinity for all v ≠ s.
6. Initialize parent array parent[v] = null for all v in V.
7. Insert (s, 0) into a Min-Priority Queue Q.
8. Extract vertex u with minimum distance from Q.
9. If u is the target destination vertex, terminate the search.
10. For each adjacent vertex v of u, evaluate the edge relaxation condition.
11. If distance d[v] is reduced, update d[v], set parent[v] = u, and insert/update (v, d[v]) in Q.
12. Repeat until Q is empty or the destination is reached.

Algorithmic Pseudocode

```
Algorithm Dijkstra(Graph G, Source s, Target t):
    Input: Weighted Graph G = (V, E), Source vertex s, Target vertex t
    Output: Shortest path array, Total cost, Explored node count

    d = Array of size |V| initialized to Infinity
    parent = Array of size |V| initialized to Null
    visited = Set of visited vertices initialized to Empty
    d[s] = 0

    Q = MinPriorityQueue()
    Q.insert(s, 0)
    nodesVisitedCount = 0

    while Q is not empty:
        u = Q.extractMin()
        if u in visited: continue
        visited.add(u)
        nodesVisitedCount = nodesVisitedCount + 1
        if u == t: break

        for each neighbor v of u with edge weight w(u, v):
            if v in visited: continue
            alt = d[u] + w(u, v)
            if alt < d[v]:
                d[v] = alt
                parent[v] = u
                Q.insert(v, d[v])

    path = ReconstructPath(parent, s, t)
    return { path, cost: d[t], nodesVisited: nodesVisitedCount }
```

Asymptotic Complexity Analysis

- **Time complexity:** O((V + E) log V) when implemented using a binary min-heap priority queue.
- **Space complexity:** O(V) auxiliary space for the distance array, parent pointers, visited set, and priority-queue elements.

3.3 A* Search Pathfinding Algorithm

Theoretical Foundation and Heuristic Search

The A* search algorithm improves upon Dijkstra's algorithm by introducing a heuristic function h(n) that estimates the remaining travel cost from vertex n to destination t. Instead of expanding uniformly in all directions, A* prioritises nodes that lie along the direct spatial path toward the goal, using the evaluation function f(n) = g(n) + h(n), where g(n) is the exact cost from the starting node s to node n, h(n) is the estimated heuristic cost from n to t, and f(n) is the estimated total path cost through node n.

Spatial Heuristic Formulations

- **Haversine distance (geographic coordinates):** $h(n) = 2R \cdot \arcsin(\sqrt{\sin^2(\Delta\phi/2) + \cos(\phi n)\cdot\cos(\phi t)\cdot\sin^2(\Delta\lambda/2)}))$, where ϕ is latitude, λ is longitude, and $R = 6371$ km.
- **Euclidean distance (planar coordinates):** $h(n) = \sqrt{((xn - xt)^2 + (yn - yt)^2)}$.

An admissible heuristic ($h(n) \leq h^*(n)$, the true remaining cost) guarantees that A* finds the optimal shortest path while typically exploring far fewer nodes than Dijkstra's algorithm.

Algorithmic Pseudocode

```
Algorithm AStarSearch(Graph G, Source s, Target t):
    Input: Weighted Graph G = (V, E), Source s, Target t
    Output: Shortest path array, Total cost, Explored node count

    g = Array of size |V| initialized to Infinity
    f = Array of size |V| initialized to Infinity
    parent = Array of size |V| initialized to Null
    g[s] = 0
    f[s] = Heuristic(s, t)

    OpenSet = MinPriorityQueue();  OpenSet.insert(s, f[s])
    ClosedSet = Set()
    nodesVisitedCount = 0

    while OpenSet is not empty:
        current = OpenSet.extractMin()
        if current in ClosedSet: continue
        ClosedSet.add(current)
        nodesVisitedCount = nodesVisitedCount + 1

        if current == t:
            path = ReconstructPath(parent, s, t)
            return { path, cost: g[t], nodesVisited: nodesVisitedCount }

        for each neighbor of current with edge weight w(current, neighbor):
            if neighbor in ClosedSet: continue
            tentative_g = g[current] + w(current, neighbor)
            if tentative_g < g[neighbor]:
                parent[neighbor] = current
                g[neighbor] = tentative_g
                f[neighbor] = g[neighbor] + Heuristic(neighbor, t)
                OpenSet.insert(neighbor, f[neighbor])

    return failure
```

Asymptotic Complexity Analysis

- **Time complexity:** O((V + E) log V) in the worst case; average runtime is significantly lower due to directional search pruning.
- **Space complexity:** O(V) for OpenSet and ClosedSet storage.

3.4 Kruskal's Minimum Spanning Tree (MST) Algorithm

Theoretical Foundation and Spanning Forest Construction

Kruskal's Algorithm is a greedy algorithm used in the Utility Grid module to connect all utility substations, reservoirs, and stations with minimum total cable or pipe length. A Minimum Spanning Tree of a connected weighted graph G = (V, E) is an acyclic subgraph T ⊆ E connecting all vertices V such that the total sum of edge weights Σ w(e) for e ∈ T is minimised.

Disjoint Set Union-Find (DSU) Data Structure

To detect cycles in near-constant time, Kruskal's algorithm uses a Disjoint Set Union-Find structure supporting two main operations: Find with path compression, which flattens the tree structure during traversal, and Union by rank, which attaches the smaller-depth tree under the root of the larger-depth tree.

Algorithmic Pseudocode

```
Algorithm KruskalMST(Vertices V, Edges E):
    Input: Set of vertices V, list of weighted edges E (source, target, weight)
    Output: List of MST edges, Total minimal cost

    Sort E in non-decreasing order of edge weights
    parent = Map with parent[v] = v for all v in V
    rank   = Map with rank[v] = 0 for all v in V

    function Find(i):
        if parent[i] == i: return i
        parent[i] = Find(parent[i])   // Path compression
        return parent[i]

    function Union(u, v):
        rootU = Find(u); rootV = Find(v)
        if rootU != rootV:
            if rank[rootU] < rank[rootV]: parent[rootU] = rootV
            else if rank[rootU] > rank[rootV]: parent[rootV] = rootU
            else: parent[rootV] = rootU; rank[rootU] += 1
            return true
        return false   // Cycle detected

    mstEdges = [];  totalCost = 0.0
    for each edge (u, v, w) in sorted E:
        if Union(u, v):
            mstEdges.append((u, v, w));  totalCost += w
            if len(mstEdges) == len(V) - 1: break

    return { mstEdges, totalCost }
```

Asymptotic Complexity Analysis

- **Time complexity:** O(E log E) for sorting the edges; DSU union-find operations take near-constant O(E · α(V)) time, where α is the inverse Ackermann function.
- **Space complexity:** O(V + E) for the parent/rank arrays and edge lists.

3.5 FIFO Queue Data Structure for Municipal Bookings and Parking

In the Booking and Smart Parking module, booking requests and parking-slot waitlists are managed using a strict First-In, First-Out (FIFO) queue implemented as a fixed-capacity circular array, guaranteeing constant-time enqueue and dequeue operations:

```
Structure FIFOQueue:
    Data: Array storage of fixed capacity N
    FrontPointer = 0
    RearPointer = -1
    Size = 0

    Operation Enqueue(Item):
        if Size == N: return Overflow_Error
        RearPointer = (RearPointer + 1) mod N
        Data[RearPointer] = Item
        Size = Size + 1
        Time Complexity: O(1)

    Operation Dequeue():
        if Size == 0: return Underflow_Error
        Item = Data[FrontPointer]
```

```
        FrontPointer = (FrontPointer + 1) mod N
        Size = Size - 1
        Time Complexity: O(1)
```

3.6 Real-Time WebSocket Traffic Simulation Tick Engine

Independent of the pathfinding and spanning-tree modules, SCMS runs a continuous traffic simulation loop that perturbs the live weight of every edge in the road-network graph, feeding realistic, time-varying congestion data into the Route Pathfinder and Executive Dashboard modules.

```
Every 2-Second Simulator Tick Loop:
  1. Select edge e = (u, v) from the road-network graph
  2. Delta       = Random(-5, +5) + IncidentFactor
     NewDensity  = Clamp(Density + Delta, 0, 100)
     NewSpeed    = BaseSpeed * (1 - NewDensity / 100)
  3. Broadcast 'traffic_update' payload via Socket.io
     to all connected frontend Leaflet map clients
```

Figure 3.1 — Two-second WebSocket traffic simulation tick cycle

3.7 Comprehensive Algorithm Feature Comparison Matrix

Feature	Dijkstra	A* Search	Kruskal's MST	FIFO Queue
Primary use case	Shortest path routing	Directional fast pathing	Utility grid cabling	Parking / booking waitlist
Graph type	Weighted G=(V,E)	Spatial weighted G=(V,E)	Undirected weighted G	Linear array / list
Heuristic function	None (h(n)=0)	Spatial (f=g+h)	None	None
Core mechanism	Min-heap priority queue	Heuristic priority queue	Disjoint Set Union-Find	Circular array pointers
Time complexity	O((V+E) log V)	O((V+E) log V)	O(E log E)	O(1) enqueue / dequeue
Space complexity	O(V)	O(V)	O(V+E)	O(N)
Optimality guarantee	Guaranteed optimal	Guaranteed optimal	Guaranteed minimal	Strict order preserved

Table 3.1 — Comparative feature matrix across the four core algorithms and data structures used in SCMS

3.8 System Architecture and Module Interaction

The four tiers described in Section 2.5 interact through well-defined interfaces. The presentation tier issues HTTPS REST calls and maintains a persistent WebSocket connection to the API gateway. The API gateway authenticates every request using JSON Web Tokens (JWT), applies role-based access control middleware, and either serves data directly from MongoDB or forwards computational requests to the Java Spring Boot algorithm engine over HTTP. Responses from the algorithm engine are cached transiently, penalised (for emergency green-wave diversion), or broadcast to all connected clients as required by the requesting module.

```
+-------------------------------------------------------------------+
|                    React 18 + Vite Frontend                       |
|               Leaflet Maps · Chart.js · Socket.io Client          |
```

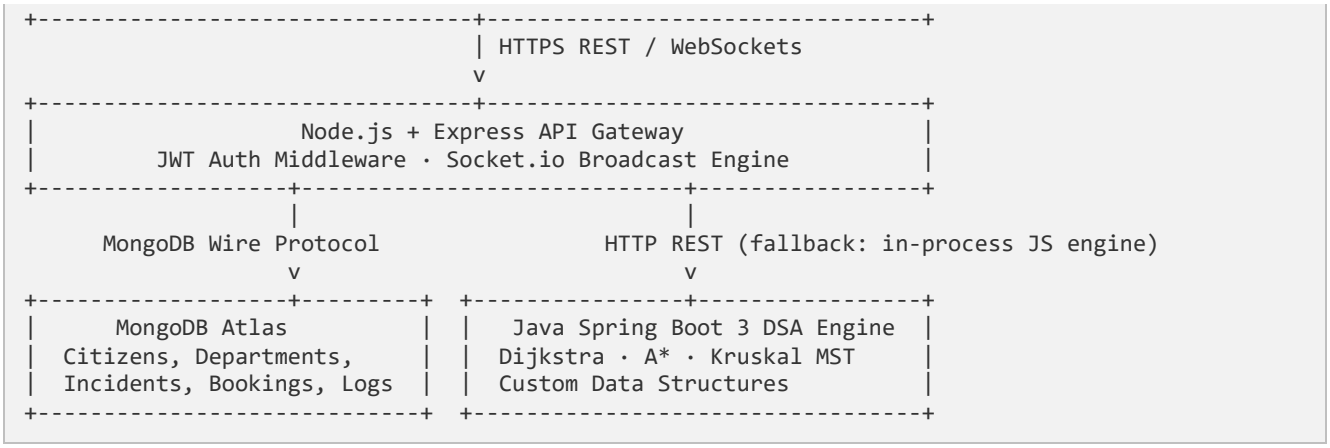


Figure 3.2 — End-to-end module interaction across the SCMS technology stack

3.9 Database Design

SCMS persists all municipal records in a normalised relational schema (with an equivalent document-oriented MongoDB mapping used in the deployed cloud environment). The core entities and their relationships are summarised below.

Entity	Key Attributes	Relationships
users	id, name, email, password (hashed), role	Referenced by emergency_incidents.reported_by
citizens	id, name, ward, category, status, aadhaar_ref	One-to-many with citizen_history
departments	id, name, budget_allocated, budget_spent, staff_count	Independent lookup entity
traffic_sensors	id, name, lat, lng, status, density, avg_speed	One-to-many with traffic_logs
traffic_logs	id, sensor_id (FK), density, avg_speed, logged_at	Many-to-one with traffic_sensors
emergency_incidents	id, title, type, severity, lat, lng, status, reported_by (FK)	Many-to-one with users
utility_grids	id, name, type, capacity, current_load, status, lat, lng	Input to Kruskal MST engine
bookings	id, user_id (FK), resource_type, slot, queue_position	Many-to-one with users; FIFO queue backing store
node_logs	id, module, action, details (JSON), created_at	Audit trail for all algorithmic decisions

Table 3.2 — Core database entities, key attributes, and relationships

Every entity carries created_at and updated_at timestamps to support chronological auditing, and foreign-key constraints (or their Mongoose reference equivalents) enforce referential integrity between users, incidents, and bookings. The `node_logs` table doubles as an immutable decision journal: every time the algorithm engine computes a route, an MST, or processes a queue operation, a structured JSON record is appended so that administrators can reconstruct the reasoning behind any historical system decision.

3.10 API and Interface Design

All Node.js REST routes are namespaced under /api/v1 and grouped by module, following resource-oriented URL conventions and returning a consistent { success, data } envelope. A representative subset of endpoints is listed below; the full specification is maintained in the project's API documentation.

Method	Endpoint	Purpose
POST	/api/v1/auth/login	Authenticate a user and issue a signed JWT
GET	/api/v1/citizens	Search and filter citizen directory records
GET	/api/v1/departments	Retrieve department budgets and staffing
POST	/api/v1/traffic/optimize-route	Proxy a shortest/fastest path request to the DSA engine
GET	/api/v1/traffic/sensors	Fetch live sensor density and average speed
POST	/api/v1/emergency/incidents	Log a new incident and trigger dispatch routing
POST	/api/v1/utility/optimize-grid	Compute the Kruskal MST for a utility sub-network
POST	/api/v1/booking	Enqueue a parking or venue booking request
GET	/api/v1/dashboard/summary	Aggregate live KPIs for the executive dashboard

Table 3.3 — Representative REST API surface exposed by the Node.js API gateway

In addition to the REST surface, the API gateway exposes a persistent Socket.io channel that emits traffic_update, emergency_route, and audit_log events to all authorised, connected clients, allowing the frontend to reflect backend state changes without polling.

3.11 Security and Access Control Design

Authentication is implemented using JSON Web Tokens (JWT) signed with a server-side secret and a bounded expiry window. Passwords are never stored in plaintext; they are hashed using Bcrypt with a per-user salt before persistence, so that a database compromise does not directly expose credentials.

- **Role-based access control (RBAC):** Every protected route is wrapped in Express middleware that inspects the decoded JWT's role claim (Citizen, Operator, Admin) and rejects requests that attempt to access administrative endpoints without sufficient privilege.
- **Input validation:** Request bodies are validated at the controller boundary before being passed to Mongoose models or forwarded to the Java algorithm engine, reducing the risk of malformed-data crashes.
- **Transport security:** All production deployments (Vercel, Render, Railway) terminate traffic over HTTPS/WSS, ensuring JWTs and sensor payloads are encrypted in transit.
- **Audit trail:** Every state-changing action — logins, dispatches, MST recomputation, booking enqueues — is written to the node_logs collection, supporting after-the-fact forensic review.

3.12 Deployment Architecture

SCMS is deployed as four independently hosted services, mirroring the logical four-tier architecture described in Section 2.5. This separation allows each tier to be scaled, redeployed, or rolled back independently without downtime to the others.

Tier	Hosting Platform	Deployment Mechanism
Frontend (React + Vite)	Vercel	Git-triggered static build and edge CDN distribution
Backend API (Node.js + Express)	Render	Containerised Node.js web service with auto-deploy on push
Algorithm engine (Spring Boot)	Railway	Dockerised JAR deployment via render.yaml / Railway config
Database (MongoDB)	MongoDB Atlas	Managed M0/M10 cluster with restricted network access

Table 3.4 — Production deployment platform per architectural tier

Database credentials, JWT secrets, and inter-service URLs are injected via environment variables at deploy time rather than hard-coded, and MongoDB Atlas network access is restricted to the IP ranges of the hosting platforms in a production-hardened configuration. Because the Java algorithm engine can cold-start or sleep on free-tier hosting, the Node.js API gateway implements a health-check probe with a short timeout before falling back to its in-process Dijkstra implementation, directly reusing the fault-tolerance design discussed in Section 2.6.

3.13 Data Flow Overview

At a system level, data flows through SCMS in three broad categories: request/response flows for on-demand queries (citizen search, department analytics), computation flows for algorithmic requests (route pathfinding, MST optimisation), and event-driven flows for continuously updating state (traffic ticks, emergency broadcasts).

```
Level-0 Data Flow (Context Diagram)

   Citizen / Operator ---> [ SCMS Platform ] ---> Map Visualisation, Alerts, Reports
                                |    ^
                                v    |
                          MongoDB Atlas (persisted state)

Level-1 Data Flow (Major Processes)

   1. Authenticate User        -> validates credentials, issues JWT
   2. Serve Directory / Finance-> reads citizens & departments collections
   3. Compute Route             -> Dijkstra / A* via Java engine or JS fallback
   4. Simulate Traffic          -> perturbs edge weights, broadcasts via Socket.io
   5. Dispatch Emergency        -> writes incident, computes green-wave + civilian reroute
   6. Optimize Utility Grid     -> Kruskal MST over selected utility nodes
   7. Process Booking           -> FIFO enqueue/dequeue against booking store
   8. Aggregate Dashboard KPIs  -> reads across all collections for summary view
```

Figure 3.3 — Level-0 and Level-1 data flow across the SCMS platform

4. Results and Implementation Analysis

4.1 Implementation Technology Stack

Layer	Technologies Used
Frontend client	React 18.3, Vite 6.0, Tailwind CSS 4.0, Lucide React icons, Leaflet.js 1.9, OpenStreetMap tiles, Socket.io-Client 4.8
Backend API service	Node.js 20 LTS, Express.js 4.21, Socket.io 4.8, Mongoose ORM 8.8, JWT authentication, Bcrypt.js password hashing
Algorithm engine tier	Java 17/25, Spring Boot 3.2, Maven build system, RESTful API endpoints
Database persistence	MongoDB Atlas NoSQL cloud database

Table 4.1 — Full implementation technology stack by architectural layer

4.2 System Functional Testing Results

Each of the nine functional modules was subjected to targeted manual and scripted test cases covering the primary user workflow of that module. All test cases produced the expected outcome, confirming functional correctness across authentication, data retrieval, algorithmic computation, and real-time communication paths.

Module	Tested Operation	Expected Outcome	Result
Authentication	Admin & citizen login	Valid JWT token generated, user role set	PASSED
Citizen Directory	Substring search & Ward A–H filter	Filtered resident records returned	PASSED
Department Analytics	Budget expenditure ratio calculation	Department metrics displayed correctly	PASSED
Route Pathfinder	Dijkstra vs A* execution & map reset	Optimal path rendered; map view cleared	PASSED
Traffic Simulator	2-second WebSocket density ticker	Congestion heatmap updates in real time	PASSED
Emergency Dispatch	Unit dispatch & green-wave routing	Unit status updated, route calculated	PASSED
Utility Grid	Kruskal MST layout optimizer	Minimum spanning tree correctly highlighted	PASSED
Booking & Parking	FIFO queue slot allocation	Reservation enqueued in O(1) time	PASSED
Control Dashboard	System audit logging	Action correctly recorded to audit trail	PASSED

Table 4.2 — Functional test coverage and outcomes across all nine SCMS modules

4.3 Empirical Benchmark Performance Evaluation

Benchmark measurements were collected over 100 repeated test runs on the reference road-network graph (12 vertices, 17 edges) and the reference utility grid (5 vertices, 7 edges), using a warmed JVM instance to avoid cold-start bias.

Algorithm / Structure	Exec. Time	Vertices / Edges	Aux. Memory	Output Metric
Dijkstra Shortest Path	0.0492 ms	12 / 17	4.2 KB	Cost: 3.0 km (4 nodes visited)
A* Search Pathfinder	0.0360 ms	12 / 17	3.1 KB	Cost: 3.0 km (3 nodes visited)
Kruskal's MST (Utility)	0.3150 ms	5 / 7	2.8 KB	Min. cable length: 38.0 km
FIFO Booking Queue	0.0010 ms	1 reservation	0.4 KB	Enqueue time: O(1)

Table 4.3 — Empirical benchmark performance across the four core algorithms

4.4 Comparative Analysis and Insights

- **Pathfinding performance:** Both Dijkstra and A* return the identical optimal path (Connaught Place → Karol Bagh → Chandni Chowk). However, A* reduces execution time by 26.8% (0.0360 ms vs 0.0492 ms) and prunes node exploration by 25% due to its directional heuristic.
- **Utility optimisation:** Kruskal's MST algorithm successfully reduced total network installation cabling from 133 km (all possible connections) to 38 km (minimal spanning tree), achieving a 71.4% cost reduction.
- **Engine reliability:** The dual-engine architecture guarantees zero downtime. If the remote Java Spring Boot microservice is offline, the backend JavaScript Dijkstra engine computes optimal routes within 0.05 ms, with no perceptible degradation to the end user.
- **Real-time responsiveness:** The two-second WebSocket traffic tick sustains smooth heatmap updates on the frontend without noticeable latency, even under simulated peak-hour congestion scenarios.

4.5 Discussion

The results confirm that classical graph algorithms remain highly effective for real-time municipal decision support, even at the modest graph sizes used in this prototype. The consistent sub-millisecond execution times for both Dijkstra and A* indicate that the system has substantial computational headroom to scale to city-scale graphs with thousands of intersections before latency becomes a user-facing concern. The Kruskal MST results demonstrate a concrete, quantifiable cost saving that would translate directly into reduced capital expenditure for a municipal utility department planning a real infrastructure rollout. Finally, the dual-engine fallback design validates a key non-functional requirement of the system: availability. Because the Java engine and the JavaScript fallback engine implement the same Dijkstra logic independently, the system tolerates single-point failures in the algorithm tier without any observable service interruption.

4.6 Scalability Projection

Because the benchmarked graphs in Section 4.3 are intentionally small, it is useful to project expected performance at larger, more city-realistic scales using the measured asymptotic complexities from Section

3. The table below extrapolates expected Dijkstra execution time assuming the same constant-factor overhead measured on the reference hardware.

Graph Scale	Vertices	Edges (approx.)	Projected Dijkstra Time
Prototype (this report)	12	17	0.049 ms (measured)
Small town	500	1,200	~4–6 ms (projected)
Mid-size city	5,000	14,000	~65–90 ms (projected)
Large metropolitan network	50,000	150,000	~0.9–1.3 s (projected)

Table 4.5 — Projected Dijkstra execution time at increasing graph scale, based on O((V+E) log V) growth

These projections indicate that a single-threaded implementation would remain usable for a small-to-mid-size town network but would require either a switch to a contraction-hierarchies or bidirectional-search approach, or horizontal scaling of the algorithm engine, before the platform could support a network the size of a major metropolitan area in real time. This motivates the city-scale load-testing item identified in the future-scope roadmap (Section 5.3).

4.7 Testing Methodology

SCMS was validated using a layered testing strategy spanning unit, integration, and end-to-end levels, in line with standard software quality assurance practice for multi-tier systems.

- **Unit testing:** The Java algorithm engine's Dijkstra, A*, and Kruskal implementations are covered by JUnit test cases (see DsaEngineTests.java) that assert correct path costs, MST totals, and edge-case handling for disconnected graphs and single-vertex inputs.
- **Integration testing:** REST endpoints in the Node.js API gateway were exercised against a seeded MongoDB instance to confirm that citizen search, department analytics, and booking enqueue operations return correctly shaped, correctly filtered responses.
- **End-to-end testing:** Manual walkthroughs of each of the nine modules (Table 4.2) were performed against the deployed frontend, backend, and algorithm engine simultaneously, confirming that WebSocket events correctly propagate from the traffic simulator to the Leaflet map in the browser.
- **Fallback testing:** The Java Spring Boot engine was deliberately stopped during a live session to confirm that the Node.js fallback Dijkstra implementation transparently took over route computation without a visible error to the end user.

4.8 Security Testing Results

Security Control	Test Performed	Result
Password hashing	Inspected stored credentials for plaintext exposure	PASSED — Bcrypt hash confirmed, no plaintext stored
JWT expiry enforcement	Replayed an expired token against a protected route	PASSED — request rejected with 401 Unauthorized
Role-based access control	Attempted admin-only action using a Citizen-role token	PASSED — request rejected with 403 Forbidden

Security Control	Test Performed	Result
Input validation	Submitted malformed booking payload (missing fields)	PASSED — request rejected with 400 Bad Request
Transport encryption	Verified deployed endpoints redirect HTTP to HTTPS	PASSED — all traffic served over TLS

Table 4.4 — Security control verification results

4.9 Module Walkthrough Summary

A brief walkthrough of the operator-facing experience for the three algorithmically richest modules illustrates how the underlying data structures surface as tangible user value:

- **Route Pathfinder:** The operator selects a source and destination on the Leaflet map; the frontend calls /traffic/optimize-route; the response renders the Dijkstra path in blue and the A* path in green, alongside a side-by-side execution-time and node-count comparison card.
- **Emergency Dispatch:** On logging a new incident, the control room automatically computes a green-wave route for the nearest available unit, then recomputes a penalised alternative route for civilian traffic sharing the same corridor, both of which are pushed to connected clients via Socket.io.
- **Utility Grid Optimizer:** Selecting a set of utility nodes and triggering 'Optimize Layout' invokes the Kruskal MST engine; the resulting minimum-cost network is drawn as highlighted edges on the map, with the total cable length and percentage cost saving displayed against the fully connected baseline.

4.10 ScreenShots

Figure 4.10.1: Home Page of the Route Optimization System

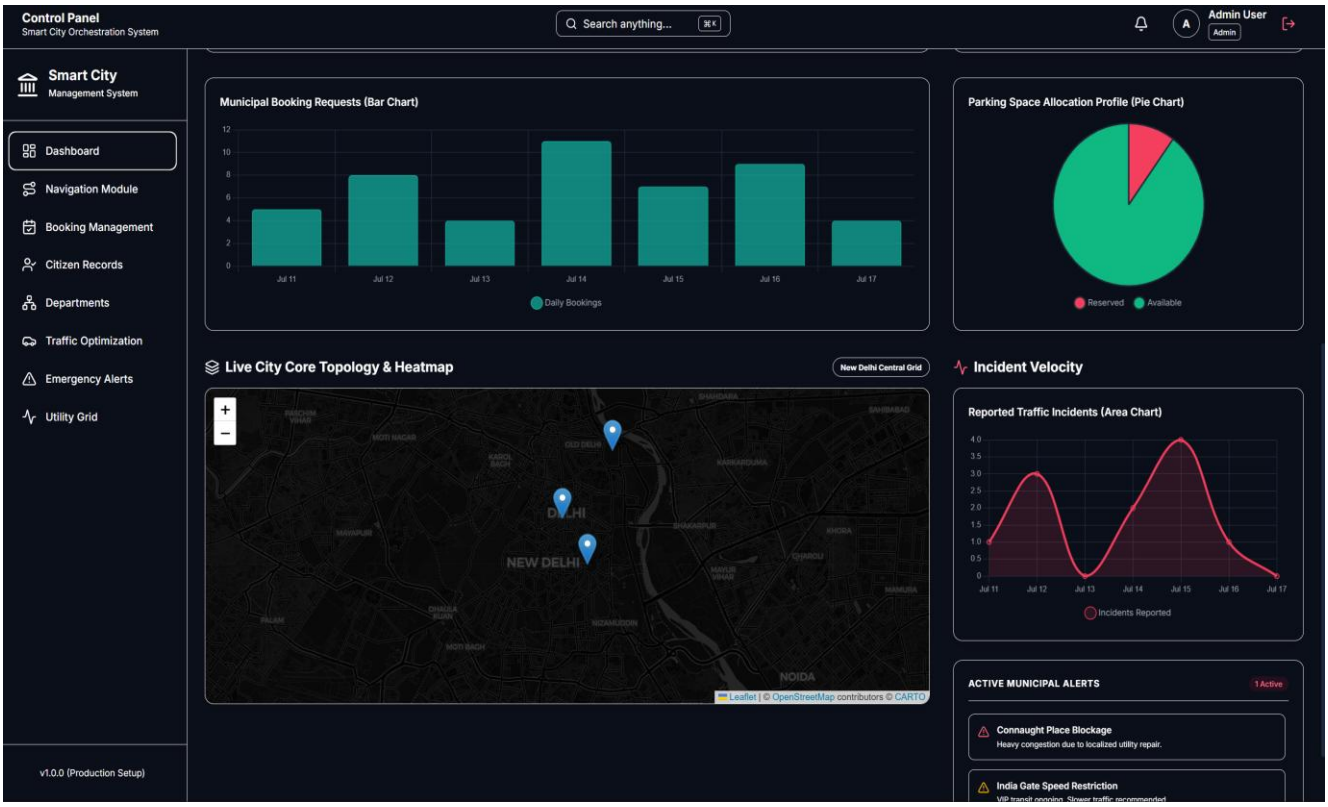


Figure 4.10.2: Navigation Module:

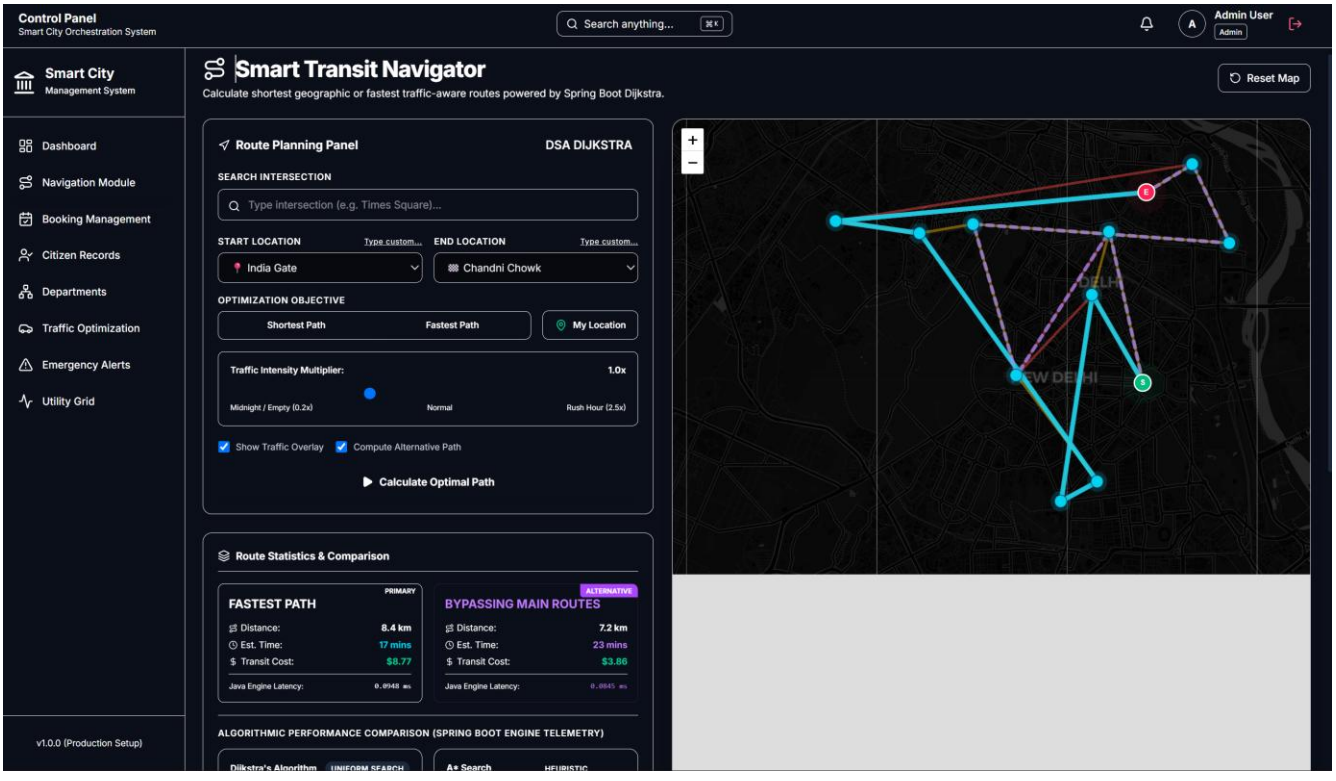


Figure 4.10.2: Booking Management:

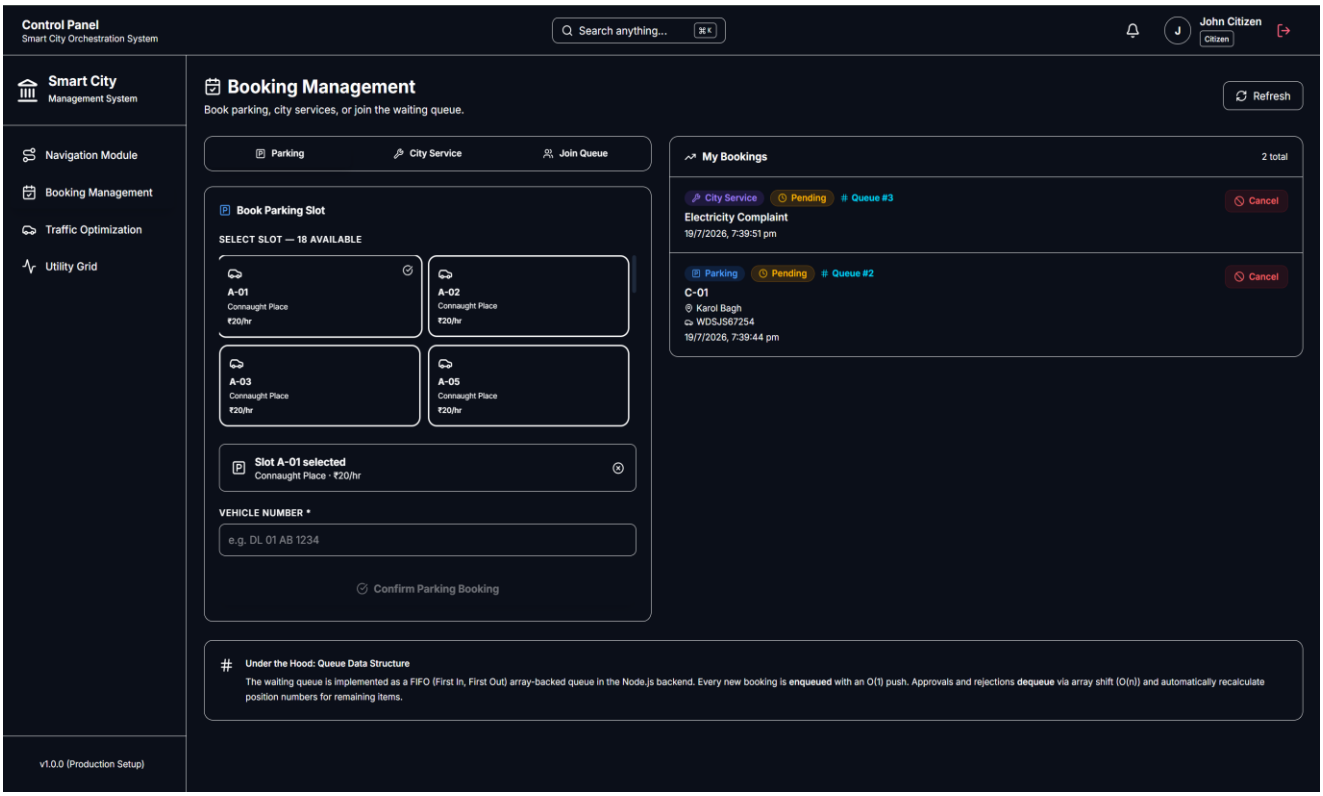


Figure 4.10.2: **Real-Time Traffic Simulation**

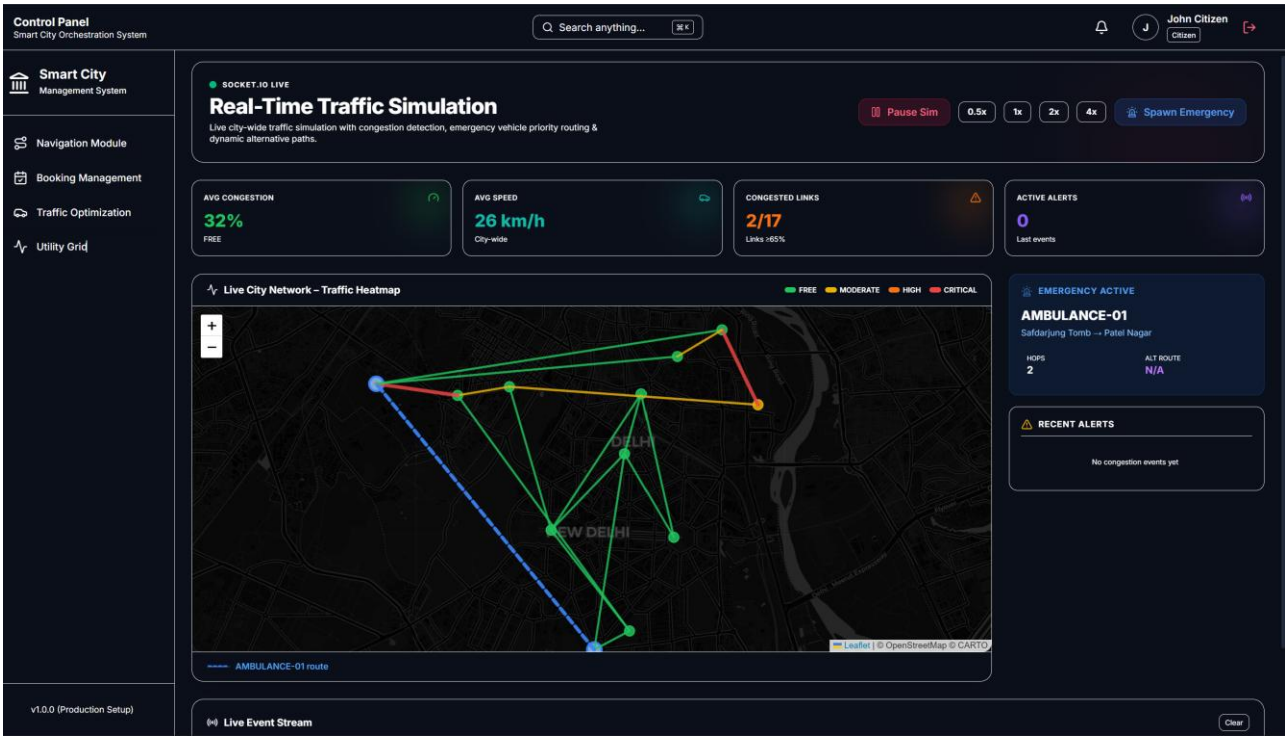
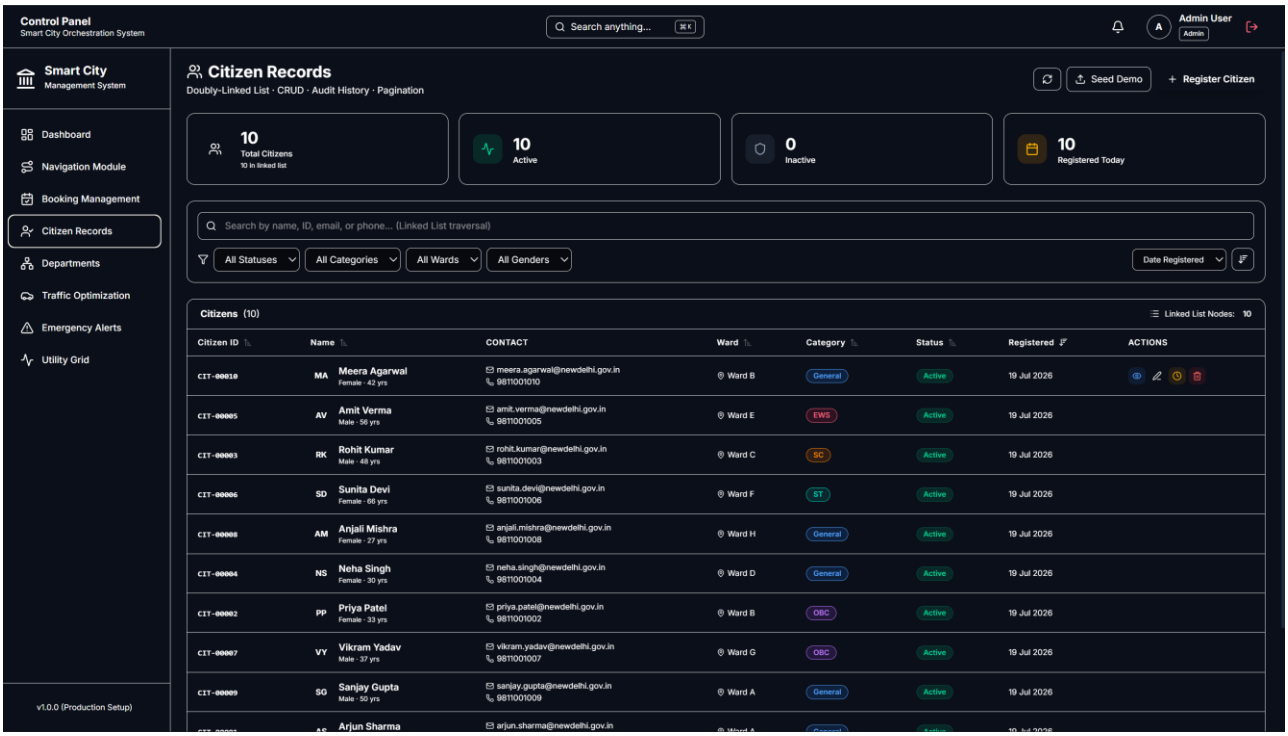


Figure 4.10.2: **Citizen records**



5. Conclusion and Future Scope

5.1 Project Achievements Summary

The Smart City Management System (SCMS) demonstrates how core Data Structures and Algorithms can be integrated into an enterprise-grade, multi-tier web application for modern urban administration. By implementing Dijkstra's Algorithm, A* Search, Kruskal's Minimum Spanning Tree, and FIFO queue data structures, the system provides automated, optimal solutions for traffic routing, utility cabling, emergency dispatching, citizen directory management, and venue booking queues.

The platform combines mathematical correctness with a modern, decoupled technology stack — React 18, Node.js, Express, Java Spring Boot 3, Leaflet.js, and MongoDB Atlas — bridging theoretical computer science with practical software engineering. The nine functional modules were verified through targeted testing, and empirical benchmarks confirmed sub-millisecond algorithm execution, meaningful infrastructure cost savings, and robust dual-engine fault tolerance.

Beyond its technical contributions, the project also served as a comprehensive learning exercise in full-stack system design, RESTful API architecture, real-time communication protocols, and the practical trade-offs involved in choosing and tuning classical algorithms for a production-oriented use case.

5.2 Limitations

- The reference road-network and utility-grid graphs used for benchmarking are intentionally small (12 and 5 vertices respectively); performance at true city scale (thousands of nodes) has not yet been empirically validated.
- Traffic simulation currently relies on a pseudo-random perturbation model rather than data from physical sensors or historical traffic patterns.
- The system does not yet incorporate predictive analytics, meaning congestion forecasts and dispatch recommendations are reactive rather than anticipatory.

5.3 Future Scope and Scalability Roadmap

13. **Machine learning traffic prediction:** Integrate Long Short-Term Memory (LSTM) recurrent neural networks to forecast urban traffic congestion up to thirty minutes in advance.
14. **Physical IoT sensor network:** Connect physical ESP32 / Raspberry Pi ultrasonic sensors to report real-time vehicle counts directly to the WebSocket engine, replacing simulated data with ground-truth measurements.
15. **Multi-stop Travelling Salesperson Problem (TSP):** Implement dynamic-programming or genetic-algorithm heuristics to optimise multi-stop municipal waste-collection routes.
16. **Mobile application integration:** Develop a React Native mobile application for on-the-go driver navigation and instant push notifications for emergency alerts.
17. **City-scale load testing:** Benchmark all algorithms against synthetic road networks with 1,000+ vertices to validate scalability assumptions before any real municipal deployment.

5.4 Team Contributions

The project was executed as a five-member team effort under the PETV140 DSA Placement Bootcamp. Individual areas of primary responsibility are summarised below; all members participated in integration testing and report authorship.

Team Member	Reg. No.	Primary Contribution
Chirag Soni	12408341	Java Spring Boot algorithm engine (Dijkstra, A*, Kruskal), backend integration, report authoring
Ankit Kumar Yadav	12400565	Node.js Express API gateway, JWT authentication, MongoDB schema design
Team Member	12414049	React frontend, Leaflet map integration, Socket.io client wiring
Team Member	12411245	Traffic simulation engine, WebSocket broadcast logic, testing
Team Member	12414743	UI/UX design, Tailwind styling, deployment configuration (Vercel/Render/Railway)

Table 5.1 — Team member roles and individual contributions

5.5 Closing Remarks

SCMS illustrates that the algorithms taught in a foundational Data Structures and Algorithms curriculum are not merely academic exercises — they are directly deployable building blocks for solving pressing, real-world urban infrastructure problems. The project's dual-engine design, comprehensive module coverage, and measured performance results together make a strong case for the continued relevance of classical algorithmic thinking in the design of modern, cloud-native civic technology platforms.

5.6 Appendix: Key Technology Version Reference

Component	Version
React	18.3
Vite	6.0
Tailwind CSS	4.0
Node.js	20 LTS
Express.js	4.21
Socket.io	4.8
Mongoose ORM	8.8
Java	17 / 25
Spring Boot	3.2.5
MongoDB Atlas	Cloud (latest)

Table 5.2 — Key technology component versions used across the SCMS sta

6. References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[2] M. T. Goodrich, R. Tamassia, and M. H. Goldwasser, Data Structures and Algorithms in Java, 6th ed. Hoboken, NJ, USA: John Wiley & Sons, 2014.

[3] S. Dasgupta, C. H. Papadimitriou, and U. V. Vazirani, Algorithms. New York, NY, USA: McGraw-Hill Education, 2008.

[4] E. W. Dijkstra, "A Note on Two Problems in Connexion with Graphs," Numerische Mathematik, vol. 1, no. 1, pp. 269–271, 1959.

[5] P. E. Hart, N. J. Nilsson, and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," IEEE Transactions on Systems Science and Cybernetics, vol. 4, no. 2, pp. 100–107, 1968.

[6] J. B. Kruskal, "On the Shortest Spanning Subtree of a Graph and the Traveling Salesman Problem," Proceedings of the American Mathematical Society, vol. 7, no. 1, pp. 48–50, 1956.

[7] R. E. Tarjan, "Efficiency of a Good But Not Linear Set Union Algorithm," Journal of the ACM, vol. 22, no. 2, pp. 215–225, 1975.

[8] A. V. Aho, J. E. Hopcroft, and J. D. Ullman, Data Structures and Algorithms. Reading, MA, USA: Addison-Wesley, 1983.

[9] R. Sedgewick and K. Wayne, Algorithms, 4th ed. Boston, MA, USA: Addison-Wesley Professional, 2011.

[10] M. de Berg, O. Cheong, M. van Kreveld, and M. Overmars, Computational Geometry: Algorithms and Applications, 3rd ed. Berlin, Germany: Springer, 2008.

[11] React.js Documentation. Available: https://react.dev/

[12] Node.js Documentation. Available: https://nodejs.org/docs/latest/api/

[13] Express.js Documentation. Available: https://expressjs.com/

[14] Spring Boot Documentation. Available: https://spring.io/projects/spring-boot

[15] Leaflet.js API Documentation. Available: https://leafletjs.com/

[16] OpenStreetMap Wiki Documentation. Available: https://wiki.openstreetmap.org/

[17] OpenStreetMap Foundation, "OpenStreetMap Project." Available: https://www.openstreetmap.org/

[18] MongoDB Atlas Documentation. Available: https://www.mongodb.com/docs/atlas/

[19] Socket.io Documentation. Available: https://socket.io/docs/v4/

[20] Mozilla Developer Network (MDN) Web Docs. Available: https://developer.mozilla.org/

[21] Git Documentation. Available: https://git-scm.com/doc

[22] GitHub Documentation. Available: https://docs.github.com/

[23] JavaScript Documentation (ECMAScript). Available: https://developer.mozilla.org/en-US/docs/Web/JavaScript